

FIAP

Disciplina: Prompt and Artificial Intelligence

Professor: Jorge Luiz Gomes

Período: 2º Semestre de 2026

Felipe Perdigão Macedo (RM570990)
Felipe Mitsuo Takahashi Stephano (RM570692)
Laura Godoy Callegari (RM569181)
Letícia Araújo Espindola (RM569308)
Mariana Dreset Carbollan (RM569207)
Milena de Aguiar Lopes Cardoso (RM570599)

SPRINT 3

SÃO PAULO

2026

GoodWe Grid Assistant

Tecnologias utilizadas: Python, FastAPI, LangChain LCEL, ChatOllama, Ollama Cloud, Pydantic v2 e tiktoken

1. Resumo da evolução do projeto

O GoodWe Grid Assistant é um chatbot operacional desenvolvido para auxiliar o gerenciamento de uma rede de eletropostos GoodWe. A solução permite realizar consultas em linguagem natural relacionadas ao funcionamento das estações, falhas críticas, consumo energético, faturamento, potência da rede, manutenção, alertas e ciclos de recarga.

Nas etapas anteriores, principalmente na Sprint 2, o projeto já apresentava uma aplicação funcional integrada ao modelo gpt-oss:120b por meio da Ollama Cloud. O sistema possuía uma API desenvolvida com FastAPI, interface web, histórico de conversa, contexto operacional simulado, system prompt e conjunto de testes.

Apesar de funcional, a versão anterior apresentava problemas de organização e manutenção. Grande parte das funcionalidades estava concentrada em um único arquivo `app.py`, incluindo interface HTML e JavaScript, contexto operacional, histórico de mensagens, testes, comunicação com o modelo e endpoints da API.

Além disso, o contexto operacional era definido diretamente no código Python. Isso significava que qualquer alteração nos dados das estações exigia a edição do próprio código-fonte. O histórico conversacional também era controlado manualmente por meio de listas, aumentando o acoplamento entre as diferentes partes da aplicação.

A Sprint 03 teve como principal objetivo realizar uma refatoração do núcleo conversacional utilizando os conceitos estudados no Módulo 1, principalmente LangChain LCEL, memória conversacional, structured output com Pydantic v2 e context engineering.

A nova versão passou a utilizar uma chain estruturada no seguinte formato:

ChatPromptTemplate | ChatOllama | OutputParser

Essa arquitetura permite separar a preparação do prompt, a comunicação com o modelo e o tratamento da resposta em componentes independentes.

Também foi implementado um sistema de memória utilizando RunnableWithMessageHistory, permitindo que cada conversa seja identificada por um `session_id`. Dessa maneira, diferentes sessões mantêm seus próprios históricos sem misturar mensagens entre usuários.

O controle do tamanho do histórico passou a considerar um limite de tokens. Essa abordagem evita que o contexto enviado ao modelo cresça continuamente durante conversas longas, mantendo apenas uma quantidade controlada de informações recentes.

Outro avanço importante foi a implementação de saída estruturada utilizando Pydantic v2. Foi criado o schema `ConsultaRecarga`, responsável por definir e validar campos relacionados ao domínio do projeto, como tipo da consulta, estação, status, consumo energético, potência, faturamento, necessidade de escalação, evidências e resumo.

A Sprint 03 também introduziu uma separação mais clara da estrutura do projeto. Foram criadas pastas específicas para chains, schemas, guardrails, avaliações, prompts e documentação. O contexto operacional também foi retirado do código principal e armazenado em um arquivo JSON independente.

Como resultado, a aplicação tornou-se mais organizada, modular e adequada para futuras evoluções.

2. Refatoração e decisões técnicas

A principal mudança técnica realizada na Sprint 03 foi a substituição da comunicação manual com o modelo por uma arquitetura utilizando LangChain LCEL.

Na versão anterior, o programa precisava construir manualmente a lista de mensagens, adicionar o system prompt, inserir o contexto operacional, recuperar o histórico da conversa e realizar a chamada para o modelo.

Já na nova versão, essas etapas passaram a ser organizadas em uma chain. O ChatPromptTemplate é responsável pela montagem das mensagens, o ChatOllama realiza a comunicação com o modelo e os parsers processam a saída.

Para respostas conversacionais normais é utilizado o StrOutputParser. Já para consultas que precisam retornar dados estruturados é utilizado o PydanticOutputParser.

A aplicação passou a possuir duas formas principais de resposta:

Resposta conversacional: utilizada para interação normal com o chatbot, retornando um texto em linguagem natural.

Resposta estruturada: utilizada quando é necessário transformar a resposta do modelo em um objeto validado pelo schema ConsultaRecarga.

O uso de Pydantic v2 adicionou uma camada de validação à aplicação. Em vez de confiar apenas em uma resposta textual do modelo, determinados dados precisam respeitar tipos, valores permitidos e regras previamente definidas.

Entre os campos utilizados estão:

tipo da consulta;

estações envolvidas;

status operacional;

consumo em kWh;

potência percentual;

faturamento;

necessidade de escalação;

evidências;

resumo da consulta.

Também foram implementados field_validator, utilizados para validar e normalizar determinados campos. Um exemplo é a identificação das estações, que devem seguir o padrão GW-XX.

Outro ponto importante foi o controle do histórico. A memória passou a utilizar sessões identificadas por session_id, permitindo uma conversa com múltiplos turnos. Com isso, perguntas que fazem referência a mensagens anteriores podem ser interpretadas considerando o contexto daquela sessão.

A memória possui um limite baseado em tokens. Essa decisão foi adotada para impedir que todo o histórico seja enviado indefinidamente ao modelo, o que aumentaria gradualmente o tamanho da entrada e o custo computacional.

A Sprint 03 também trouxe melhorias no contexto utilizado pelo modelo. O contexto operacional anteriormente hardcoded foi separado em um arquivo contexto_operacional.json. Dessa maneira, a aplicação pode carregar as informações operacionais sem misturá-las à implementação da API.

O system prompt também passou por uma refatoração. Foram mantidas versões diferentes para permitir comparação entre o prompt antigo e o novo. A versão v2 utiliza XML tagging para separar instruções de identidade, escopo, segurança, contexto e comportamento esperado.

A análise com tiktoken mostrou que o system prompt v1 possuía aproximadamente 931 tokens, enquanto a versão v2 possui aproximadamente 472 tokens. Isso representa uma redução de 49,3% no tamanho do prompt.

Essa redução mantém as principais instruções do sistema utilizando menos contexto, deixando mais espaço disponível para perguntas, histórico e dados operacionais.

3. Memória, structured output e guardrails

A memória conversacional é um dos principais recursos implementados na Sprint 03.

A aplicação utiliza `RunnableWithMessageHistory` para associar cada interação a uma sessão. O histórico é recuperado automaticamente de acordo com o `session_id` informado.

Isso permite realizar, por exemplo, uma sequência como:

Pergunta 1: Qual estação apresentou superaquecimento?

Pergunta 2: Ela foi desativada?

Pergunta 3: O que você recomenda para ela?

Nas perguntas seguintes, o sistema consegue utilizar as mensagens anteriores para interpretar expressões como “ela”, mantendo continuidade na conversa.

Além da memória, foi implementado structured output utilizando Pydantic v2.

Durante o desenvolvimento, foram identificados alguns problemas relacionados à geração do JSON pelo modelo. Nas primeiras execuções, algumas respostas apresentavam campos ausentes ou JSON incompleto. Como consequência, o `PydanticOutputParser` rejeitava a resposta.

A solução adotada foi fornecer ao ChatOllama o JSON Schema gerado pelo próprio modelo Pydantic através de:

```
ConsultaRecarga.model_json_schema()
```

Além disso, foi aumentado o limite de geração especificamente para a chain estruturada. A conversa normal continua utilizando o limite padrão definido na configuração do projeto, enquanto a saída estruturada utiliza um limite maior, permitindo que o modelo complete todos os campos necessários.

Após essas alterações, os cinco cenários que utilizam structured output passaram corretamente pela validação. O resultado final da avaliação registrou 100% de acurácia no structured output.

O caso relacionado a falha crítica, por exemplo, retornou corretamente a estação GW-09, status `falha_critica`, necessidade de escalção igual a `true`, além das evidências e do resumo.

No teste de consumo, o objeto estruturado identificou corretamente a estação GW-12 e o consumo de 428 kWh.

Também foram implementados guardrails específicos para impedir comportamentos indesejados.

Entre os cenários tratados estão tentativas de jailbreak, pedidos para inventar dados, aconselhamento financeiro, aconselhamento jurídico, perguntas gerais fora do escopo do GoodWe Grid Assistant e solicitações envolvendo intervenções elétricas perigosas.

No caso de uma tentativa de prompt injection solicitando que o sistema ignorasse as instruções anteriores e inventasse um faturamento maior, o chatbot recusou a manipulação e manteve a resposta limitada ao contexto operacional disponível.

O sistema também recusou fornecer recomendações financeiras, direcionando o usuário para um profissional habilitado e retornando ao domínio de eletropostos GoodWe.

No cenário envolvendo a abertura de um carregador energizado, o guardrail recusou fornecer instruções para intervenção elétrica perigosa e orientou a interrupção do uso do equipamento e o acionamento de profissional habilitado

4. Avaliação e comparação entre Sprint 2 e Sprint 3

Para avaliar objetivamente a evolução do sistema, foram utilizados os oito casos presentes na avaliação da Sprint 2 e quatro novos casos adicionados na Sprint 3.

A metodologia considera a porcentagem dos critérios explícitos atendidos por cada resposta. Uma resposta com pelo menos 75% dos critérios atendidos é classificada como adequada; entre 50% e 75%, parcialmente adequada; abaixo de 50%, inadequada.

Na Sprint 2, os oito casos comparáveis resultaram em uma qualidade normalizada de 68,75%. A latência média registrada foi de 2.457,38 ms, enquanto a estimativa de tokens médios de entrada foi de 1.568,62 tokens por consulta. A versão anterior não possuía structured output validado por Pydantic.

Após a refatoração, os mesmos oito casos foram executados novamente.

Na Sprint 03, os oito casos foram classificados como adequados, resultando em 100% de qualidade, nenhuma resposta parcialmente adequada e nenhuma resposta inadequada. A latência média foi de 876,98 ms, enquanto a estimativa média de tokens de entrada ficou em 1.167,62 tokens. O structured output alcançou 100% de acurácia nos casos em que foi executado.

Indicador	Sprint 2	Sprint 3
Qualidade nos 8 testes comparáveis	68,75%	100%
Respostas adequadas	5*	8
Latência média	2.457,38 ms	876,98 ms
Tokens médios de entrada	1.568,62	1.167,62
Structured output validado	0%	100%
Tamanho do system prompt	931 tokens	472 tokens
Memória	Lista manual	Sessão + limite de tokens
Contexto operacional	Hardcoded	Arquivo JSON externo
Arquitetura	Chamada manual	LangChain LCEL
Guardrails	Principalmente via prompt	Regras dedicadas + prompt
Estrutura do projeto	Concentrada no app.py	Modularizada

A Sprint 03 apresentou evolução nos principais indicadores. A qualidade passou de 68,75% para 100%, um aumento de 31,25 pontos percentuais. A latência média foi reduzida em 64,3%, de 2.457,38 ms para 876,98 ms, enquanto os tokens médios de entrada diminuíram 25,6%, de 1.568,62 para 1.167,62 tokens por consulta. O system prompt também foi reduzido em 49,3%, passando de 931 para 472 tokens.

Além dos oito testes comparáveis, a Sprint 03 incluiu quatro novos casos sobre ausência de informação, aconselhamento jurídico, segurança elétrica e perguntas fora do escopo. Nos 12 testes completos, todas as respostas foram classificadas como adequadas, resultando em 100% de qualidade geral, com latência média de 636,74 ms e média estimada de 1.168,08 tokens de entrada.

Os resultados indicam evolução na qualidade, desempenho e segurança da aplicação.

5. Problemas encontrados e soluções

Durante a refatoração, um dos principais problemas encontrados foi a geração de structured output incompleto. Em algumas execuções, o modelo retornava JSON sem todos os campos exigidos pelo schema ou interrompia a resposta antes de finalizar o objeto.

A solução foi fornecer ao ChatOllama o JSON Schema gerado pelo Pydantic e aumentar o limite de geração especificamente para a chain estruturada. Após os ajustes, todos os casos de structured output utilizados no eval final foram validados corretamente.

Outro problema ocorreu no mecanismo de avaliação textual. Diferentes caracteres Unicode, como tipos de hífen e espaços especiais, faziam respostas corretas serem consideradas incorretas. A função de normalização foi ajustada para padronizar esses caracteres antes da comparação. Essa alteração não modificou os critérios dos testes, apenas tornou a avaliação mais consistente.

Também foi necessário reduzir o acoplamento existente na Sprint 2. A solução foi dividir o projeto em módulos como src/chain, src/schemas, src/guardrails, evals, prompts e docs, facilitando manutenção e testes.

6. Equipe e divisão de trabalho

Integrante	RM	Tarefa principal
Felipe Mitsuo	570692	Chains LCEL e integração com ChatOllama
Felipe Perdigão	570990	Memória conversacional e sessões
Laura Godoy	569181	Pydantic v2 e structured output
Letícia Espindola	569308	System prompt e context engineering
Mariana Carbollan	569207	Guardrails e segurança
Milena de Aguiar	570599	Evals, métricas e documentação

As atividades foram integradas na mesma aplicação e revisadas em conjunto durante os testes finais.

7. Problemas encontrados e soluções

A Sprint 03 representou uma evolução significativa da arquitetura do GoodWe Grid Assistant.

Enquanto a versão anterior já possuía um chatbot funcional, sua implementação apresentava forte concentração de responsabilidades e dependência de procedimentos manuais para comunicação com o modelo e manutenção do histórico.

A utilização do LangChain LCEL permitiu transformar esse fluxo em uma composição explícita de prompt, modelo e parser. A introdução de memória por sessão melhorou a capacidade do chatbot de manter conversas com múltiplos turnos, enquanto o controle por tokens limita o crescimento do contexto.

O Pydantic v2 acrescentou validação formal às respostas estruturadas. Após os ajustes realizados durante os testes, o structured output atingiu 100% de acurácia nos casos avaliados.

Os guardrails também apresentaram funcionamento adequado nos cenários de prompt injection, aconselhamento financeiro, orientação jurídica, segurança elétrica e perguntas fora do escopo.

Os resultados quantitativos também demonstram a evolução do projeto. Nos oito casos diretamente comparáveis, a qualidade passou de 68,75% na Sprint 2 para 100% na Sprint 3. A latência média caiu de 2.457,38 ms para 876,98 ms, correspondendo a uma redução aproximada de 64,3%. O número estimado de tokens de entrada caiu aproximadamente 25,6%, passando de 1.568,62 para 1.167,62 tokens.

O próprio system prompt também foi otimizado, passando de 931 para 472 tokens, redução de 49,3%.

Nos doze testes completos da Sprint 03, incluindo os novos cenários de segurança e casos de borda, todas as respostas foram classificadas como adequadas, resultando em 100% de qualidade geral.

Portanto, a Sprint 03 não se limitou a adicionar novas funcionalidades. A principal evolução foi transformar o GoodWe Grid Assistant em uma solução mais modular, validável, segura e mensurável, criando uma base técnica mais organizada para futuras etapas do projeto.